Chapter 7: Conditional Statements

Every program so far has executed its statements in sequence until it either finished or stopped early: `exit` and a failed `assert` both request termination, using the `(value, status)` pair from the last chapter to get there. That is different from what this chapter adds. Short-circuit `and` and `or` already skip evaluating an operand when the other side already decides the result, but that is a property of a single expression, not a choice about which statements to run. This chapter adds that choice: picking which statements execute at all, based on a condition checked while the program runs, without stopping it. That choice is **conditional selection**, one of the handful of control-flow constructs every general-purpose language provides. (1)

```
if (temperature > 90) {
    print("Too hot");
} else {
    print("Fine");
}
```

Exactly one branch of an `if` statement runs. The other is not merely skipped in the sense of producing no output: it is never evaluated at all.

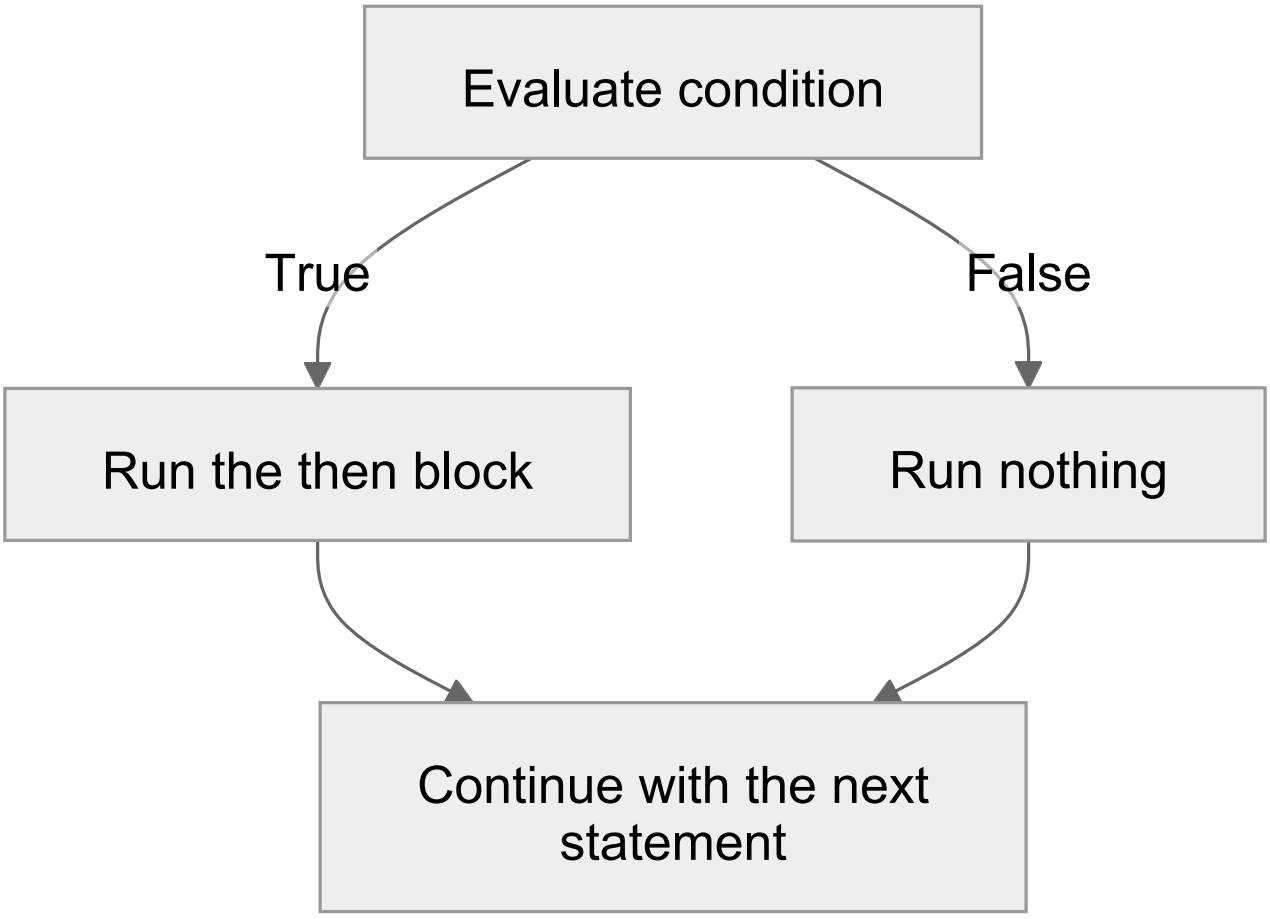
Breaking Changes

Earlier use	Change
A variable, or any identifier, named `if` or `else`	Rename it; both names are now keywords.
A program using `{` or `}` for any other purpose	Not previously legal syntax; now reserved for blocks.

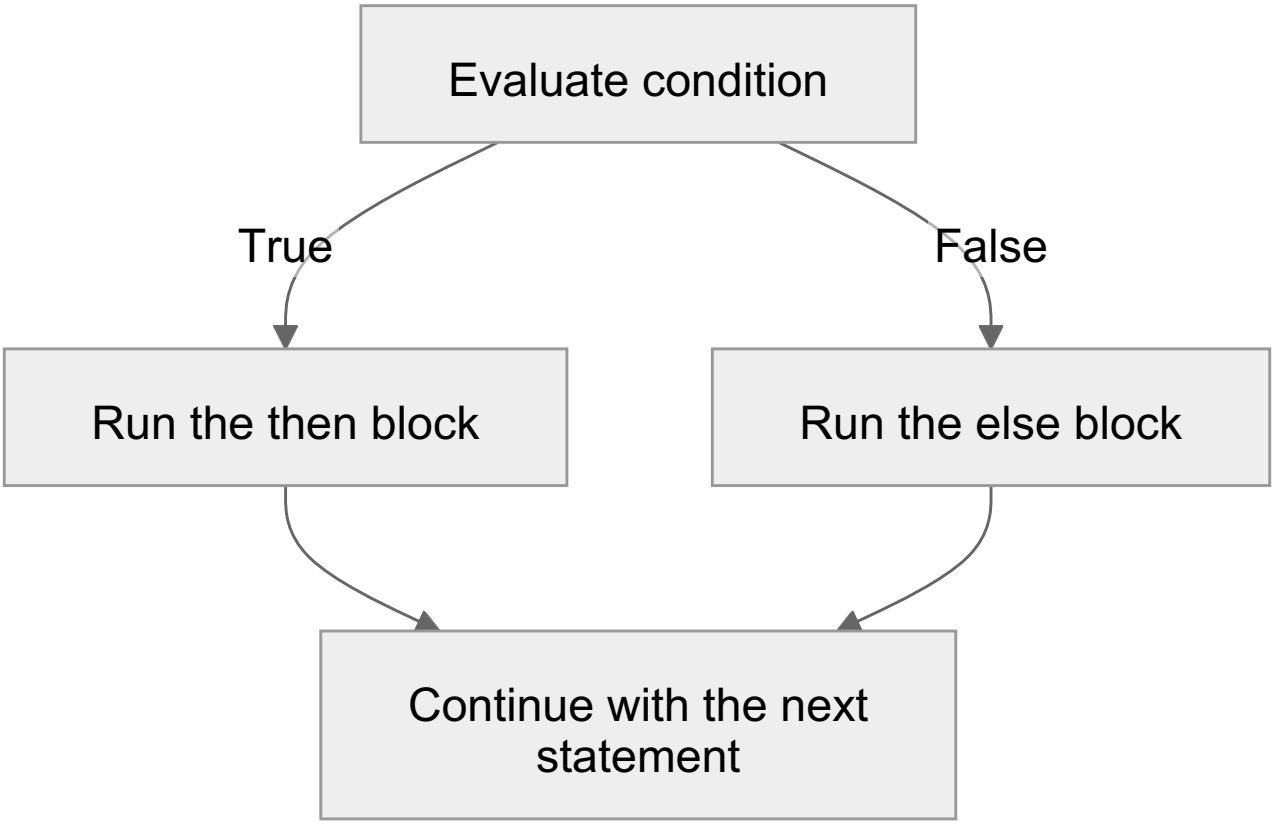
Nothing about expressions, `exit`, `assert`, or the `(value, status)` pair changes. A conditional is a new kind of statement, built entirely on top of last chapter's contract: it evaluates a child, checks the status, and only then decides what to do next.

Only the Selected Branch Runs

An `if` statement evaluates its condition once. A true condition runs the `then` branch. Without an `else`, a false condition runs nothing at all:



Add an `else`, and a false condition has somewhere to go instead of skipping straight past:



Either way, exactly one path out of the condition is taken. The branch that is not selected is never evaluated:

```
if (false) {
    exit(9);
    print(number(input()));
} else {
    print("only this branch runs");
}
```

This prints `only this branch runs` and nothing else. The unselected branch's `exit(9)` never fires, and `input()` never reads anything, even though the code sits right there in the source. This is the same discipline that already governs short-circuit `and`/`or`: an operand is not merely "not used," it is never asked to produce a value.

A condition must be a boolean, the same requirement `not`, `and`, and `or` already enforce. `if (1) { ... }` raises a type error rather than treating `1` as true.

Every Branch Is a Block

A branch is never a single bare statement. It is always a **block**: a statement list wrapped in braces.

```
if_statement ::= "if" "(" expression ")" block [ "else" block ]
block ::= "{" [ statement_list ] { ";" } "}"
```

Requiring braces everywhere avoids a well-known ambiguity in this grammar family: given `if (a) if (b) x=1; else y=2;`, it is not obvious from the text alone which `if` the `else` belongs to. (2) Vertex sidesteps the question instead of resolving it with a rule to memorize. `parse_if_statement` checks for a following `else` right after it finishes parsing its own `then` block, before returning control to whatever called it. An `else` always belongs to the `if` whose `then` block just closed, because that is the only `if` that is still in a position to claim it.

One consequence is that this grammar has no "else if" shortcut. Chaining conditionals means writing a nested `if` inside its own braces:

```
if (grade >= 90) {
    letter = "A";
} else {
    if (grade >= 80) {
        letter = "B";
    } else {
        letter = "C or lower";
    };
};
```

The inner `if` is just an ordinary statement that happens to live inside the outer `else` block. Nothing new was added to support it.

A block is not a new kind of node. It reuses the `statement_list` node already built for the whole program:

```
def parse_block(tokens):
    # block ::= "{" [ statement_list ] { ";" } "}"
    # A block wraps a statement_list in braces, so running it does not differ
    # from running the top-level program's statement_list, including
    # accepting a trailing semicolon before the closing brace. Unlike the
    # top-level program, the statement_list itself is optional: "{}" is an
    # empty block, not an error. Extra semicolons are harmless even when
    # there are no statements. The top-level program is unchanged.
    tokens = require(tokens, "{", "Expected '{' to start a block")
    while tokens[0]["tag"] == ";":
        tokens = tokens[1:]
    if tokens[0]["tag"] == "}":
        return {"tag": "statement_list", "statements": []}, tokens[1:]
    statements, tokens = parse_statement_list(tokens)
    tokens = require(tokens, "}", "Expected '}' to close a block")
    return statements, tokens
```

Reusing `parse_statement_list` this way gives it a second caller. Until now it only ever had one: the top-level program, which always ends at the true end of input. A block ends at `}` instead, so recognizing a trailing semicolon needs to extend to that terminator too:

```
if tokens[0]["tag"] in (None, "}"):
    break
```

This is new coverage, not a correction: `parse_statement_list` was already right about the only case it had ever needed to handle. `{ x=1; }` and `{ x=1 }` both parse the same way, the same as a trailing semicolon at the end of a program always has.

A block still differs from the top level in one way: its `statement_list` is optional. `{ }` is an empty block, not an error, so `if (found) { }` is a legal way to say "do nothing" for a condition worth naming but not yet worth acting on. `{ ; }` and `{ ;;; }` are also empty blocks: extra semicolons remain harmless even without a statement. `parse_block` skips leading semicolons before checking for `}`. If the block is empty, it returns an empty statement list without calling `parse_statement_list`. Otherwise, that function handles the statements and any remaining extra semicolons. This does not change the top-level grammar, which still requires a statement.

An `if` statement follows the same separator rule as any other statement, and `else` follows a rule of its own:

```
if (true) { x=1 };
y=2;
```

The semicolon after the closing `}` is required here, exactly as it would be after an assignment: an `if` statement is a statement, and `parse_statement_list` still needs `;` between any two statements it parses, whatever they end with. An `else`, by contrast, must follow its `then` block's closing `}` with nothing in between:

```
if (true) { x=1 } else { x=2 };
```

`parse_if_statement` looks for `else` immediately after parsing the `then` block. A semicolon there is not optional punctuation the parser skips over; it is itself the start of a new, empty-looking statement boundary, so `if (true) { x=1 }; else { x=2 }` is a syntax error: parsing stops after the `if` statement at the semicolon, and `else` is left over as the beginning of a statement that does not exist.

Parsing an If Statement

```
def parse_if_statement(tokens):
    # if_statement ::= "if" "(" expression ")" block [ "else" block ]
    # Both branches require braces, so there is no dangling-else ambiguity to
    # resolve: this function checks for "else" immediately after parsing its
    # own then-block, before returning. An "else" always belongs to whichever
    # "if" statement's then-block just closed.
    tokens = require(tokens, "if", "Expected 'if'")
    tokens = require(tokens, "(", "Expected '(' after 'if'")
    condition, tokens = parse_expression(tokens)
    tokens = require(tokens, ")", "Expected ')' after if condition")
    then_block, tokens = parse_block(tokens)
    else_block = None
    if tokens[0]["tag"] == "else":
        else_block, tokens = parse_block(tokens[1:])
    return {"tag": "if", "condition": condition, "then": then_block,
            "else": else_block}, tokens
```

The resulting tree has four parts: a condition expression, a `then` block, and an optional `else` block, which is `None` when absent.

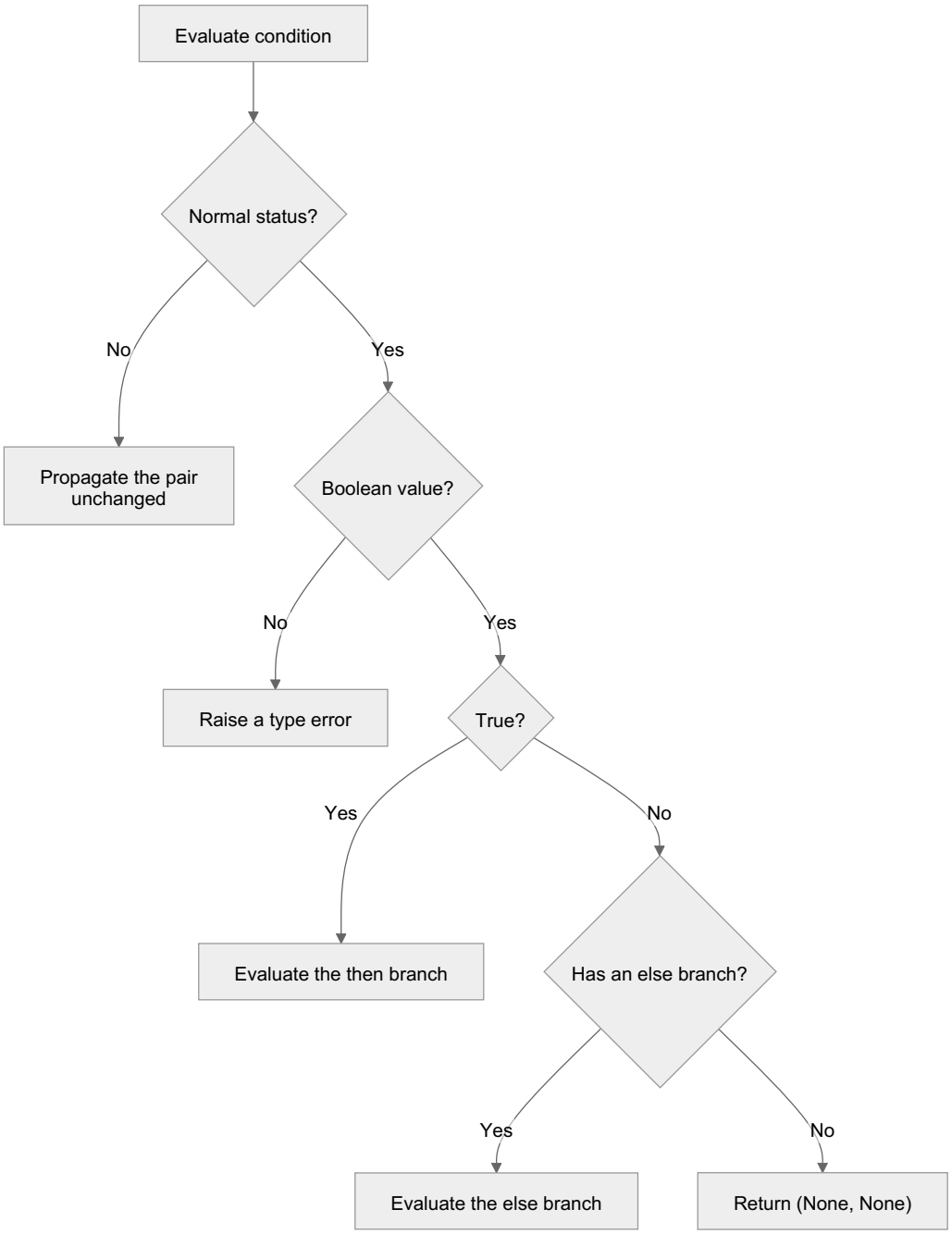
```
{
    "tag": "if",
    "condition": {"tag": ">=", "left": {...}, "right": {...}},
    "then": {"tag": "statement_list", "statements": [...]},
    "else": None,
}
```

Both `then` and `else`, when present, are ordinary `statement_list` nodes. The evaluator does not need a separate case for "block"; it already knows how to run a `statement_list`.

Evaluating a Conditional

```
if ast["tag"] == "if":
    condition, status = evaluate(ast["condition"], environment)
    if status is not None:
        return condition, status
    if type(condition) is not bool:
        raise TypeError("if condition must be a boolean expression")
    # The branch not taken is never evaluated: it cannot read input,
    # exit, or fail an assertion. Both branches share this environment;
    # a block introduces no scope of its own.
    if condition:
        return evaluate(ast["then"], environment)
    if ast["else"] is not None:
        return evaluate(ast["else"], environment)
    return None, None
```

This is four cases, in order: the condition itself escaped with a status; the condition was not a boolean; the condition was true; the condition was false. Whatever `evaluate` returns for the selected block becomes the `if` statement's own return value. There is no fifth case that inspects the block's result: an `if` statement does not consume a status, even its own branch's. It only decides which block gets to run.



The two flow diagrams earlier show what runs. This one shows the full decision the evaluator makes to get there, including the two ways an `if` statement can end without running either block at all: an escaped status, or a false condition with no `else`.

Blocks share the enclosing environment. `if (true) { x=1 }` assigns `x` in whatever environment the `if` statement itself was evaluated in, the same as an unconditional `x=1;` would. This chapter does not introduce scoping; that is a later topic.

Status Still Propagates Through Blocks

`exit` and a failed `assert` behave exactly as they did last chapter, even from inside nested conditionals:

```
x = 1;
if (true) {
    if (true) {
        exit(7);
    };
};
x = 2;
```

The innermost `exit(7)` produces `(7, "exit")`. The inner block's `statement_list` sees a non-`None` status on its first (and only) statement and returns immediately. The inner `if` returns whatever its `then` block returned. The outer block does the same, and so does the outer `if`. By the time the program's own top-level `statement_list` sees this result, it stops there too: the final `x = 2;` never runs, and `x` is left at `1`.

No node along that chain needed to know it was inside a conditional. Every one of them was already written to check `is not None` and return early; `if` simply gave them more places to be nested inside.

Testing the Contract

`test_conditionals.py` covers single-branch and two-branch conditionals, multi-statement blocks, nested conditionals (including an else-if chain written with explicit braces), the boolean-condition requirement, empty blocks, and the two propagation properties above: the unselected branch never touches input, `exit`, or an assertion, and a status from inside nested blocks escapes all of them and stops whatever statement would have run next. Parser tests separately confirm that a bare, unbraced branch is always a syntax error, that single-statement and nested braced blocks both parse, and that `{ }`, `{ ; }`, and `{ ;;; }` all parse to an empty `statement_list`.

Considerations

1. Why does the unselected branch need to be skipped entirely, rather than evaluated and merely discarded?
2. What would go wrong if `if` consumed a status returned by its own selected branch instead of propagating it unchanged?
3. This chapter's grammar has no "else if" shortcut. What would adding one change about how `else` attaches to a preceding `if`?
4. If blocks introduced their own scope in a later chapter, what would have to change about how `if` evaluates its branches?

Practice

Answers to the practice questions follow this section.

1. Write `if (x > 0) { sign = 1 } else { sign = -1 };` and trace it by hand for `x = 5` and `x = -5`. Which block runs in each case?
2. Rewrite a three-way "else if" chain (positive, negative, zero) using only `if`, `else`, and nested braces. Compare it against the grade-letter example in this chapter.

3. Predict the output and exit code of `if (true) { print(1); exit(2); print(3); } else { print(4); }`, then run it and check.
4. Find every place in `evaluator.py` where a status check happens before an `if` statement was added, and explain why `if` did not need to change any of them.

Answers to the Practice Questions

1. For `x=5`, the condition is true and the first block assigns `sign=1`. For `x=−5`, the else block assigns `sign=−1`. Only the selected block is evaluated. This two-way example assigns `−1` for zero too.
2. Nest a second conditional inside the else block:

```
x = 0;
if (x > 0) {
    sign = 1;
} else {
    if (x < 0) {
        sign = -1;
    } else {
        sign = 0;
    };
};
print(sign);
```

   This prints `0`. Changing `x` to a positive or negative number gives `1` or `−1`. The grade example's else-if spelling expresses the same selection through a chain; nesting makes the second decision explicit.
3. Output is one line, `1`, and the process exits with code `2`. `print(3)` is skipped because exit propagates out of the selected block. The else block is skipped because its condition was not selected.
4. The earlier statement-list and expression rules already pass a child's non-normal status outward before continuing. An if statement follows that same contract: evaluate its condition, check status and boolean type, then return the selected block's result. Its caller does not need to know whether the status originated in an if block or elsewhere.

References

1. Wikipedia contributors. "Control flow." Wikipedia. https://en.wikipedia.org/wiki/Control_flow
2. Wikipedia contributors. "Dangling else." Wikipedia. https://en.wikipedia.org/wiki/Dangling_else